

PAZZLE — Reconstruction Strategy

Restoring a shuffled & degraded 24×24 jigsaw to beat the 0.40 SSIM leader — the plan, the evidence, the dead-ends, and how to contribute.

Team working document · 2026-07-08 · metric = mean SSIM (skimage, channel_axis=2, data_range=255, win=7)

0 · TL;DR

The score factorises: $SSIM \approx \text{placement-quality} \times \text{restoration-quality}$ — and placement is the entire game. Perfectly placing the degraded fragments already gives ≈ 0.44 SSIM; the current leader sits at **0.40**, i.e. they have essentially *solved the puzzle* and barely restored. So we win by (1) recovering the arrangement almost perfectly, then (2) a cheap denoise on top. Everything hinges on **fragment-compatibility scoring** being strong enough for a 576-piece assembly — that is the one hard problem, and where all effort should go.

1 · The task

- **Geometry:** each 480×480 image is cut into a **24×24 grid of 20×20 px fragments** = 576 pieces.
- **Corruption:** fragments are **shuffled** to random positions, and each is **independently degraded**: brightness ± 30 , contrast 0.70–1.30, Gaussian noise σ 40–55, 3×3 blur, JPEG q35–50. (No rotation — position only.)
- **Goal / metric:** output the restored clean 480×480 image; scored by **mean SSIM** vs the clean target.
- **Submission:** exactly 700 PNGs (RGB, 480×480), named like the test files, zipped at the root.

2 · The measured landscape (this is why the strategy is what it is)

All numbers below are *measured* on held-out train images (we recover their true arrangement by Hungarian-matching degraded fragments to the clean target, giving exact labels + honest local evaluation).

Configuration	mean SSIM	meaning
Shuffled input, unchanged	0.08–0.11	doing nothing
Leader (public board)	0.40	the bar to beat
Perfect placement, degraded (no restore)	0.44	ceiling if we only solve the puzzle
Perfect placement + NLM denoise	0.57	reachable target with a trivial restorer
Perfect placement + our RestoreNet	0.4385	⚠ currently <i>below</i> no-restore — model is broken

Read this table carefully. The leader (0.40) is just under the perfect-placement ceiling (0.44), so they solved placement and gained almost nothing from restoration. But perfect placement + a *dumb* NLM denoise reaches **0.57** — a wide-open gap above the leader. The whole prize is: **get placement almost perfect, then denoise.**

3 · Pipeline architecture

576 shuffled fragments → **pairwise compatibility scorer** (learned) → **assembly solver** (best-buddies / SA) → assemble 24×24 → **restore** (NLM denoise) → output

The structure is right and should not be inverted — you cannot restore what is not yet assembled. What matters is the *strength* of each stage. Stage 1 (compatibility) is the bottleneck; stage 4 (restore) is a solved, cheap win via NLM.

4 · The one hard problem: compatibility scoring

For each ordered pair of fragments we need a score of "does *B* plausibly sit to the right of / below *A*". A learned net (**PairwiseNet**) sees both fragments straddling the seam (3, 20, 40) → a logit, trained with InfoNCE (true neighbour vs in-image distractors).

Why it is hard here

The degradation (heavy noise + JPEG + per-fragment brightness/contrast shifts) **destroys pixel-level edge continuity**. A textured 20×20 tile carries little signal once corrupted. The net learns higher-level texture compatibility, but only weakly.

Where the v1 scorer stands (measured)

Metric (full 576×576 matrix, on val)	v1	what's needed
InfoNCE val acc@48	0.477	—
True-neighbour R@1	≈0.20	→ 0.7+
True-neighbour median rank / 576	7–18	→ 1–2
Best-buddy precision (mutual top-1 correct)	≈0.48	→ ~0.9

The result: with these scores the solver returns essentially a **random** arrangement (place-accuracy ≈ 1/576, SSIM ≈ 0.10). Even the most confident pairs (best-buddies) are only coin-flips (0.48), and a jigsaw solver needs ~0.9 to grow an assembly without compounding errors.

It is NOT a solver bug

- **Oracle test:** feed the solver perfect scores → it reconstructs at **100%**. The mechanics are fine.
- Add a little noise to perfect scores → the solver finds an arrangement with *higher* total objective but **0%** placement. So the global-sum objective is **non-robust** to imperfect scores (many wrong arrangements out-score the true one).
- A best-buddies growth solver on the *real* scores reaches only 0.11 neighbour-accuracy — because the scores themselves are too weak.

Conclusion: the lever is score strength, not the solver. (A better solver — best-buddies + loop constraints — still matters, but only once scores improve.)

5 · Dead-ends we have already ruled out (please don't re-spend time here)

Idea	Verdict	Evidence
Classical edge/gradients compatibility (MGC)	✗ dead	best-buddy precision $\approx$ 0.05 (vs 0.48 learned) — degradation kills pixel continuity
Denoise fragments <i>before</i> matching	✗ no help	NLM-denoised fragments give the same weak matching — the lost info isn't just noise
Per-fragment brightness/contrast normalization	✗ no help	bb-precision barely moves (0.05 $\rightarrow$ 0.05–0.13)
Diffusion model for restoration	✗ wrong metric	SSIM rewards smoothing (NLM $h_{12} > h_8$); diffusion hallucinates HF detail $\rightarrow$ lower SSIM. And restoration isn't the bottleneck anyway.

6 · The levers we ARE pursuing

Placement (the priority) — a much stronger scorer

- **Seam-aware architecture:** the v1 net global-average-pooled its feature map, throwing away *where* the seam is. v2 keeps the spatial layout (flatten), adds GroupNorm, widens to $C=96$ (1.95M $\rightarrow$ 5.6M params).
- **Train on REAL degraded data** (`real_prob 0.6`): v1 trained on synthetic distortion only; we have 7000 real correctly-ordered images (via the recovery cache) — training on them closes the synthetic $\rightarrow$ real domain gap.
- **Harder negatives:** more in-image distractors per anchor (M 16 $\rightarrow$ 32).
- **Longer training** on both GPUs (see §7).
- **Success gauge:** a solver-independent diagnostic (`diag_scores.py`) reports best-buddy precision / R@1 — the number that actually predicts assemblability. Target: bb-precision 0.48 $\rightarrow$ $\sim$ 0.9.
- **Next, if scores improve:** replace global-sum simulated annealing with a **best-buddies greedy-growth** solver (Paikin–Tal / Pomeranz) + loop-consistency; SA only as final polish.

Restoration — cheap, solved

Use **Non-Local-Means denoise** (`--nlm`): a guaranteed 0.447 $\rightarrow$ $\sim$ 0.57 lift on well-placed images. Our learned RestoreNet is currently undertrained (near-identity) and can be revisited later to beat NLM, but it is **not on the critical path**.

7 · Suggested setup & how to run (adapt to your own environment)

General recommendations — plug in your own accounts, keys and paths.

Environment

- Organise the code as a small package and run scripts from its directory.
- Keep the dataset and model checkpoints **off your code drive** (any spare disk / scratch space) — they are large.
- A single modern GPU is enough to iterate; a free cloud GPU session (e.g. two mid-range GPUs for

Multi-GPU — a caution

Naïve data-parallel training was found to be *slower* than a single GPU for this small model + pair-batch: the per-step replication / gather overhead exceeds the compute saved. If you have several GPUs, prefer running **independent scorers concurrently** (one per GPU, different seeds) and **ensemble** their score matrices — this uses the hardware efficiently *and* lifts best-buddy precision.

~12h) comfortably trains the scorer. Use whatever compute *you* have access to.

- Watch training live with any experiment tracker (loss / accuracy / ETA per stage) using your own account and API key.
- The permutation-recovery cache (true arrangements of train images) is worth building once and reusing — it drives both real-data training and honest local evaluation.

Typical commands (illustrative — flags are the levers, not fixed values)

- Train the scorer: `train_pair.py --steps 10000 --nA 48 --M 32 --real_prob 0.6`
- Score-quality gauge: `diag_scores.py` → reports best-buddy precision / R@1 (the number to watch).
- Placement / end-to-end SSIM: `eval_place.py --full_pair` · `eval_full.py --full_pair --nlm`
- Build the 700-image submission: `infer.py --full_pair --nlm`

8 · Open problems & ideas to try (teammates welcome)

- **Push bb-precision to ~0.9** — the make-or-break number. Bigger/longer scorer, hard-negative mining, richer seam context, self-supervised pretext tasks on fragments.
- **Robust assembler** — best-buddies growth with loop constraints, trimming-and-refill, or global optimisation that ignores absolute score magnitude (rank-based).
- **Exploit structure** — known 24×24 grid, all pieces present, no rotation. A set-to-grid transformer that sees all 576 fragments and predicts positions jointly could beat pairwise greedy.
- **Beat NLM restore** — a properly trained MS-SSIM+L1 U-Net should exceed 0.57; verify and productionise (secondary priority).

PAZZLE — Pasha883's idea candidate. Bottom line for anyone picking this up: **the competition is a 576-piece jigsaw problem, not a restoration problem.** Make the compatibility scorer strong enough that best-buddy precision approaches 0.9, wire it to a growth-based solver, denoise with NLM, and you clear 0.40 with room to spare. Everything else has been measured and is documented above.